

CPSC 375 High-Performance Computing

Lecture 25

Parallel Sorting Algorithms I

Potential Speedup in Sorting

- $O(n \log n)$ optimal for any sequential sorting algorithm without using special properties of the numbers.
- Best we can expect based upon a sequential sorting algorithm using p processors is
$$O(n \log n) / p = O(\log n) \quad \text{if } p = n$$
- Has been obtained but the constant hidden in the order notation extremely large.

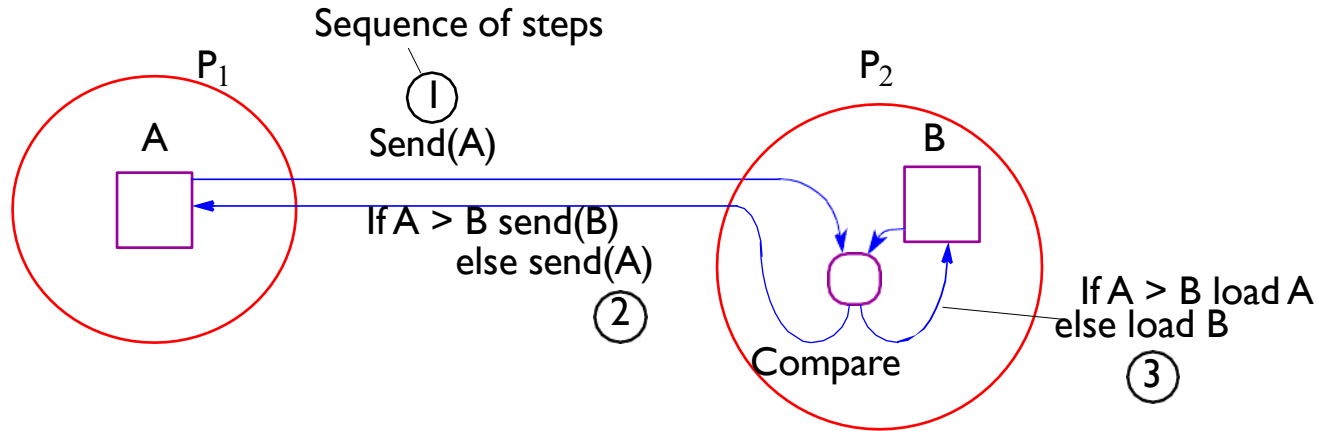
Compare-and-Exchange Sorting Algorithms

- Form the basis of several, if not most, classical sequential sorting algorithms.
- Two numbers, say A and B , are compared and exchanged if necessary:

```
if (A > B) {  
    temp = A;  
    A = B;  
    B = temp;  
}
```

Message-Passing Compare and Exchange

- P_1 sends A to P_2 , which compares A and B and sends back B to P_1 .
- If A is larger than B (otherwise it sends back A to P_1):



Message-Passing Compare and Exchange

Process P_1

send(&A, P_2);

recv(&A, P_2);

Process P_2

recv(&A, P_1);

if ($A > B$) {

 send(&B, P_1);

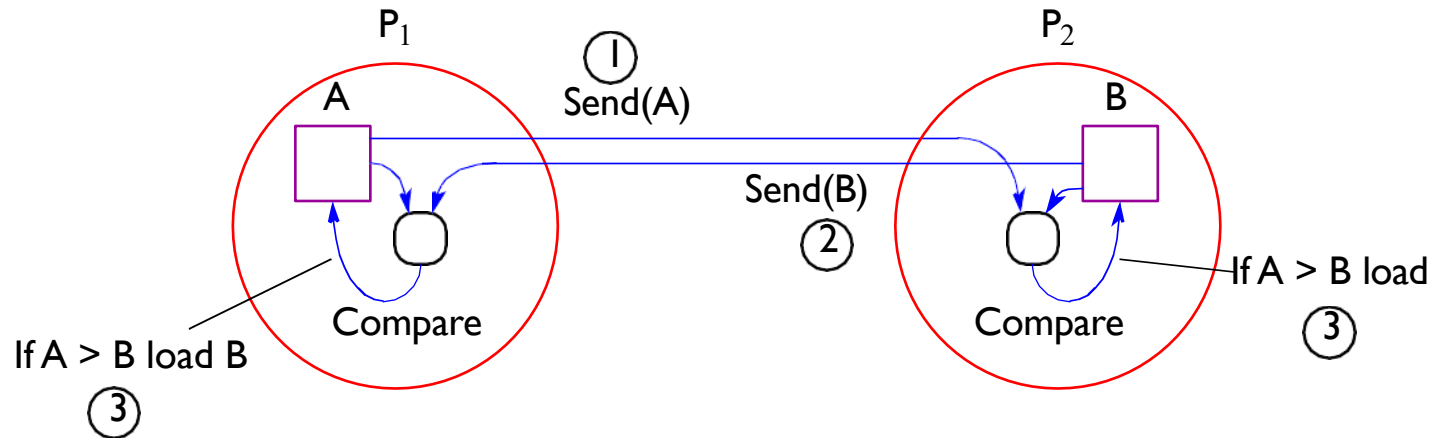
$B = A$;

} else

 send(&A, P_1);

Alternative Message Passing Method

- For P_1 to send A to P_2 and P_2 to send B to P_1 .
- Then both processes perform compare operations.
- P_1 keeps the larger of A and B and P_2 keeps the smaller of A and B :



Alternative Message Passing Method

Process P_1

```
send(&A,  $P_2$ );  
recv(&B,  $P_2$ );  
if ( $A > B$ )  $A = B$ ;
```

Process P_2

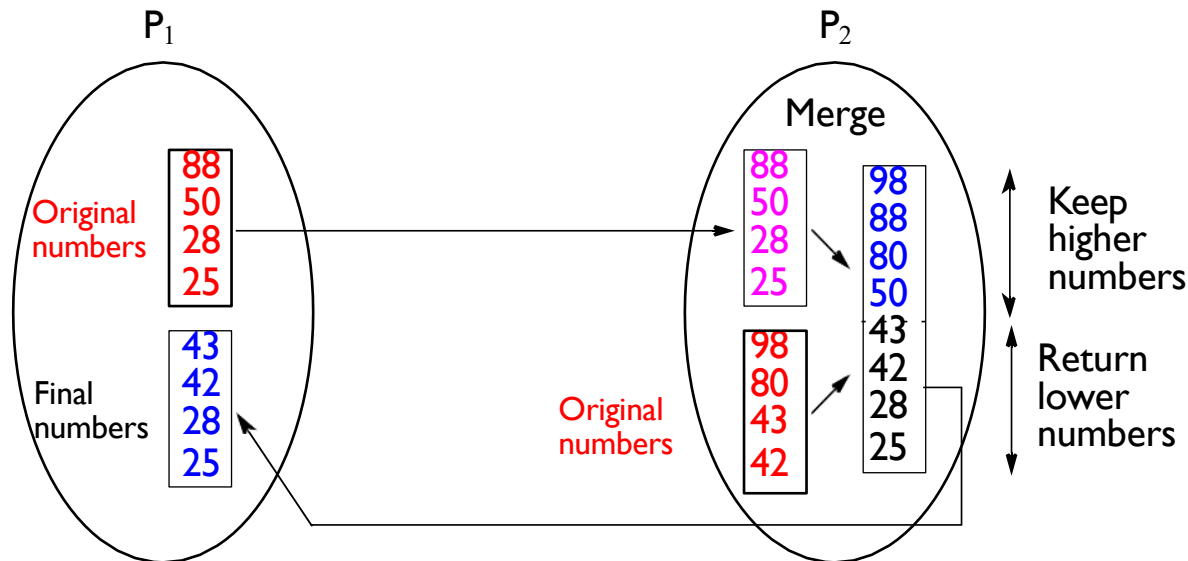
```
recv(&A,  $P_1$ );  
send(&B,  $P_1$ );  
if ( $A > B$ )  $B = A$ ;
```

Note on Precision of Duplicated Computations

- Previous code assumes that the if condition, $A > B$, will return the same Boolean answer in both processors.
- Different processors operating at different precisions could conceivably produce different answers if real numbers are being compared.
- This situation applies to anywhere computations are duplicated in different processors to reduce message passing, or to make the code SPMD.

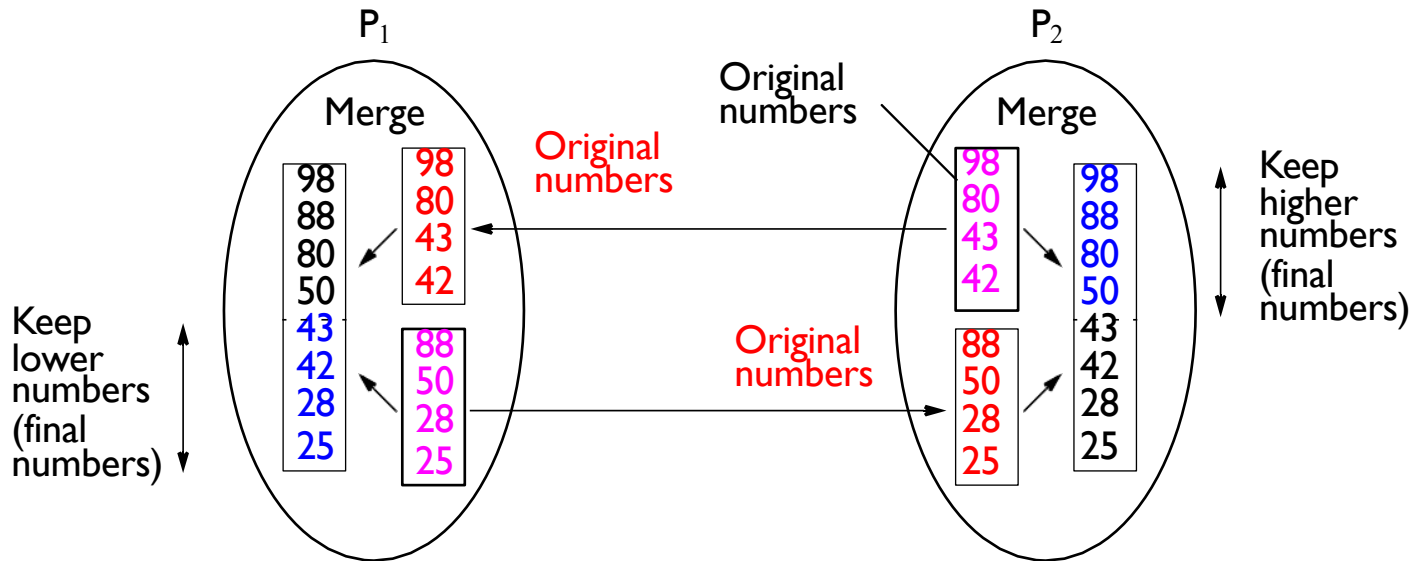
Data Partitioning

- p processors and n numbers.
- n/p numbers assigned to each processor:



Alternative Data Partitioning

- Merging two sublists



General Strategies

- Maintain a sorted list in each processor and merge the stored list with the incoming list, deleting either the top half or the bottom half of the merged list.
 - Assume that each processor will produce the same results and divide the list exactly the same way.
- Merging requires $2(n/p)$ steps to merge two lists each of n/p numbers and two message transfers.
- Merging two sorted lists is a common operation in sorting algorithms.

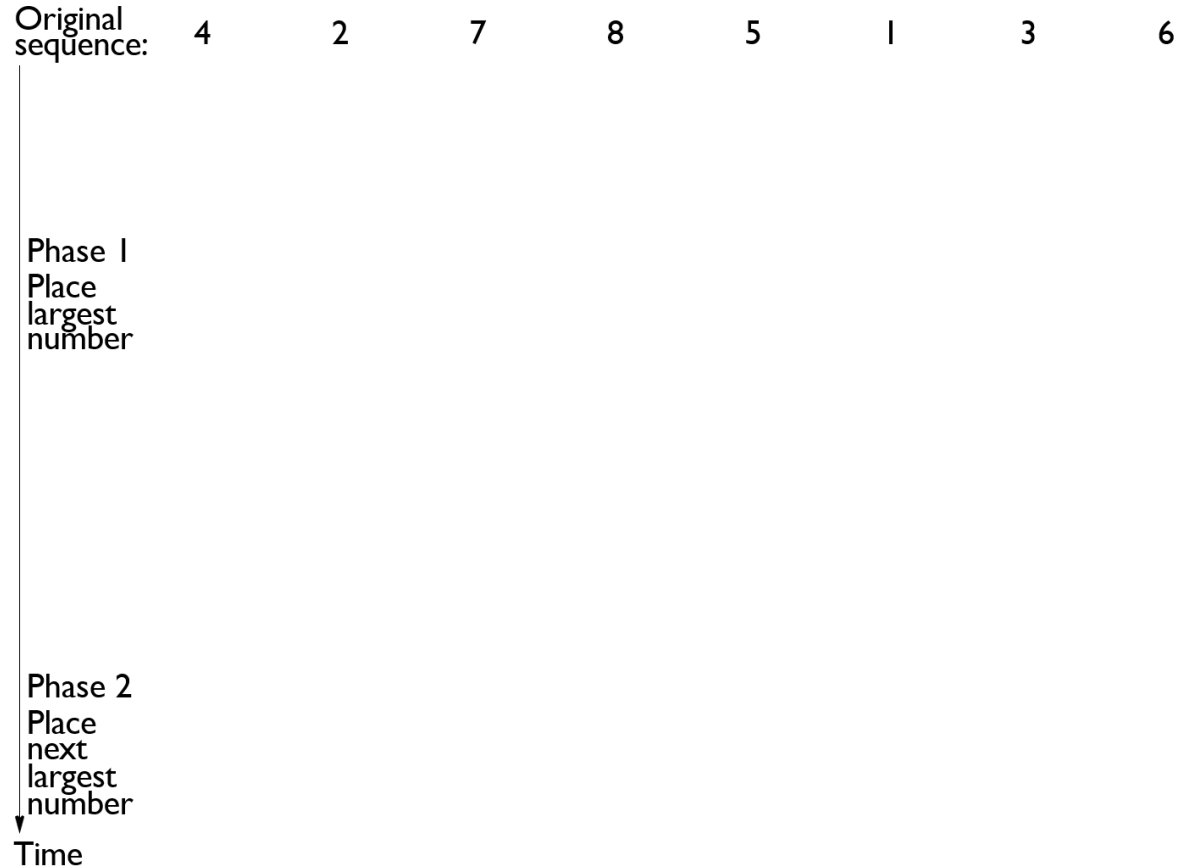
Bubble Sort

- First, largest number moved to the end of list by a series of compares and exchanges, starting at the opposite end.
- Actions repeated with subsequent numbers, stopping just before the previously positioned number.
- In this way, the larger numbers move (“bubble”) toward the end.

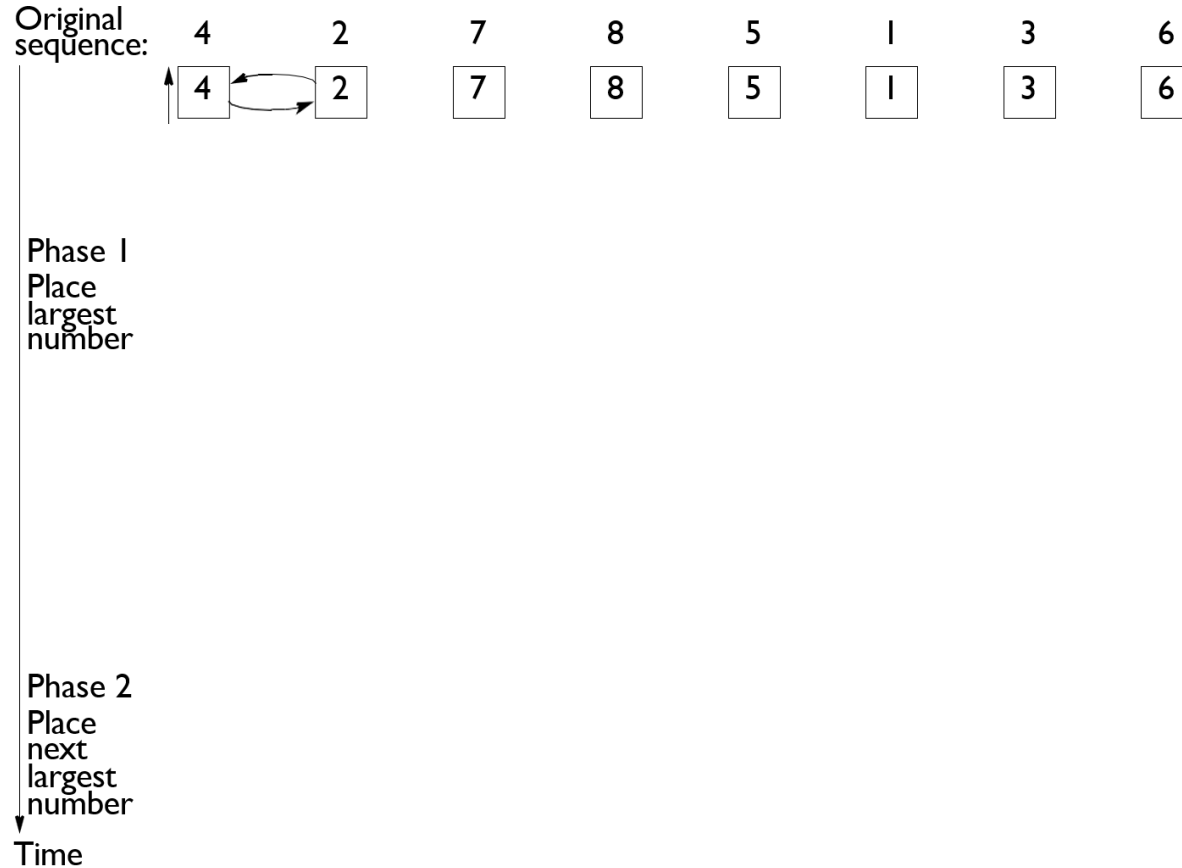
Sequential:

```
for (i = n - 1; i > 0; i--)  
    for (j = 0; j < i; j++) {  
        k = j + 1;  
        if (a[j] > a[k]) {  
            temp = a[j];  
            a[j] = a[k];  
            a[k] = temp;  
        }  
    }
```

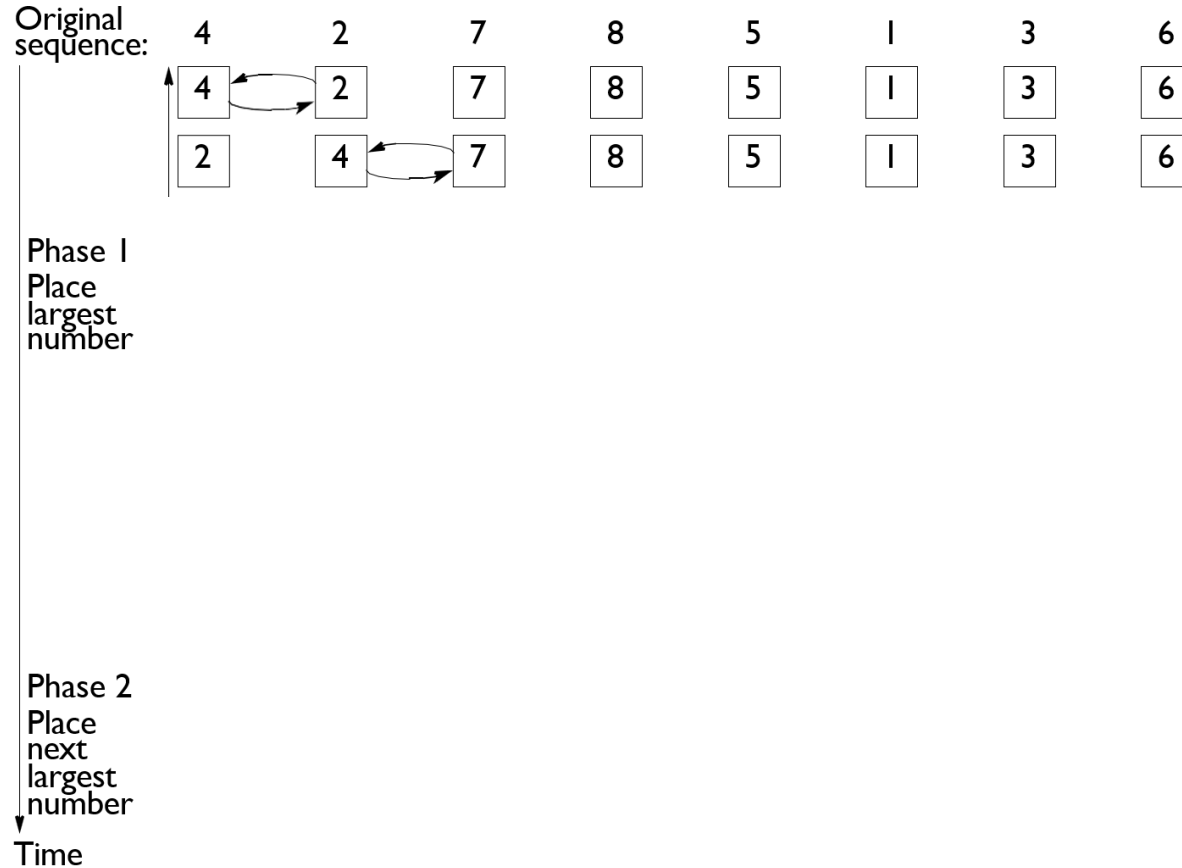
Bubble Sort



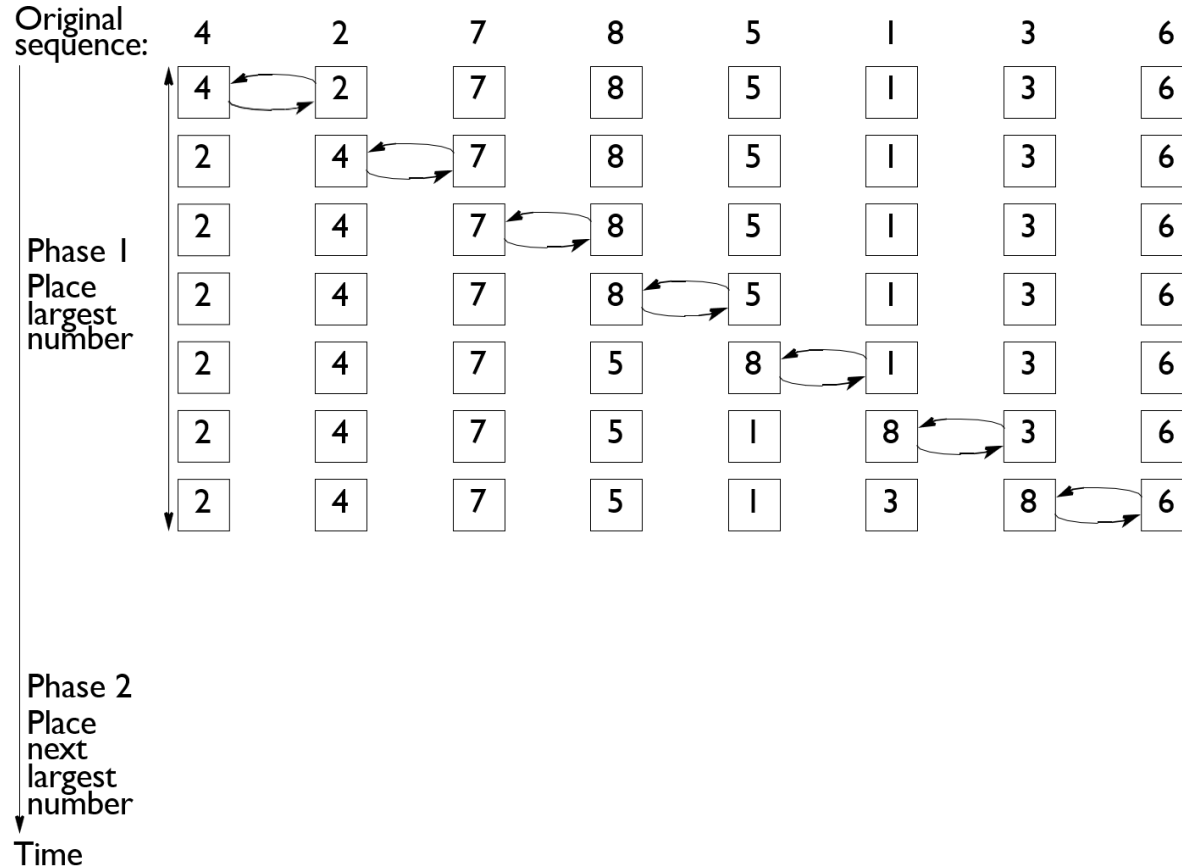
Bubble Sort



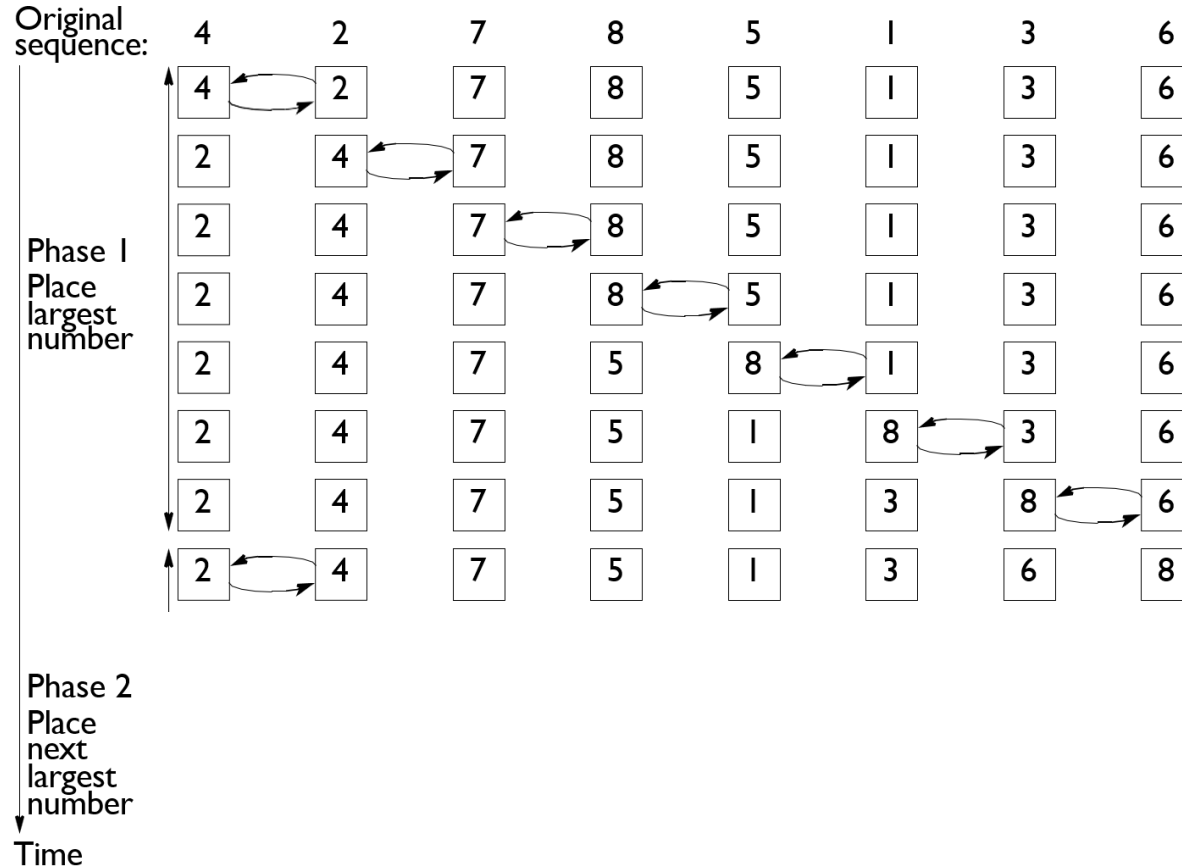
Bubble Sort



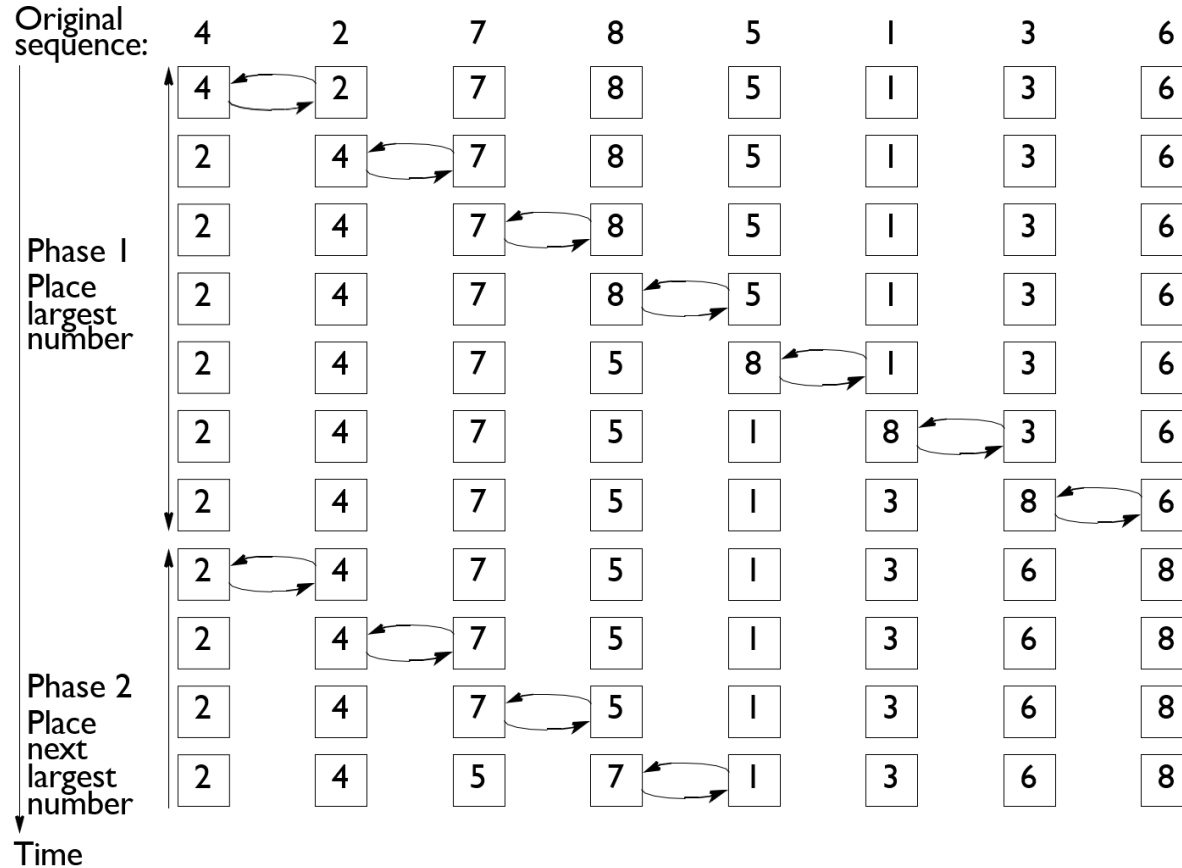
Bubble Sort



Bubble Sort



Bubble Sort



Time Complexity

Number of compare and exchange operations $= \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$

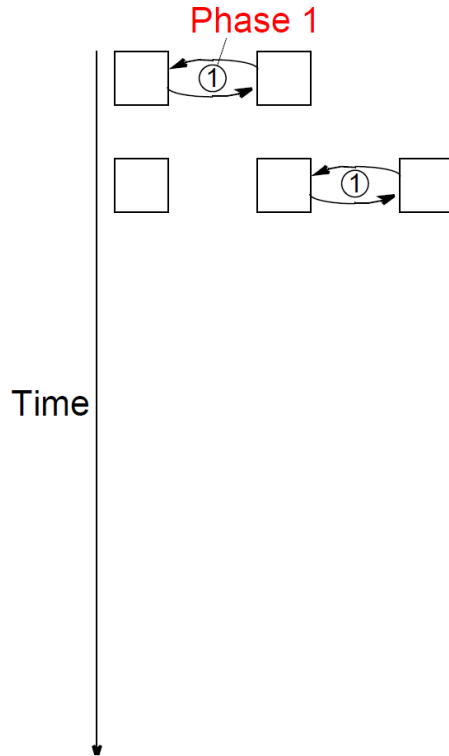
Indicates a time complexity of $O(n^2)$ given that a single compare-and-exchange operation has a constant complexity, $O(1)$.

Parallel Bubble Sort

Iteration could start before previous iteration finished if does not overtake previous bubbling action:

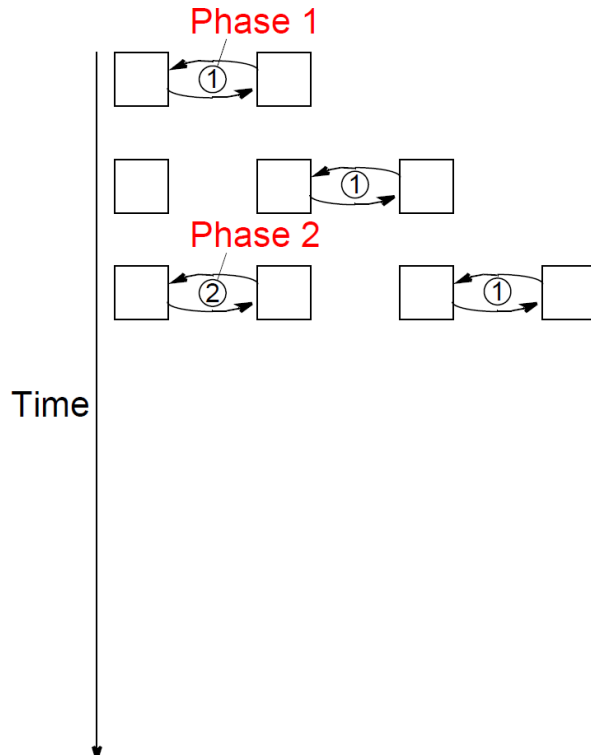
Parallel Bubble Sort

Iteration could start before previous iteration finished if does not overtake previous bubbling action:



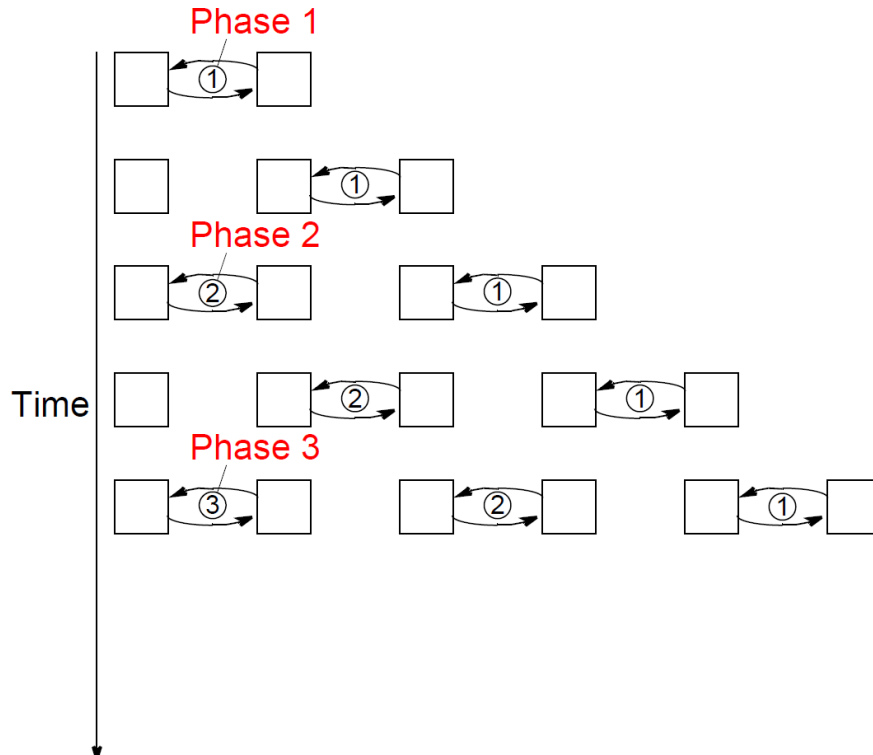
Parallel Bubble Sort

Iteration could start before previous iteration finished if does not overtake previous bubbling action:



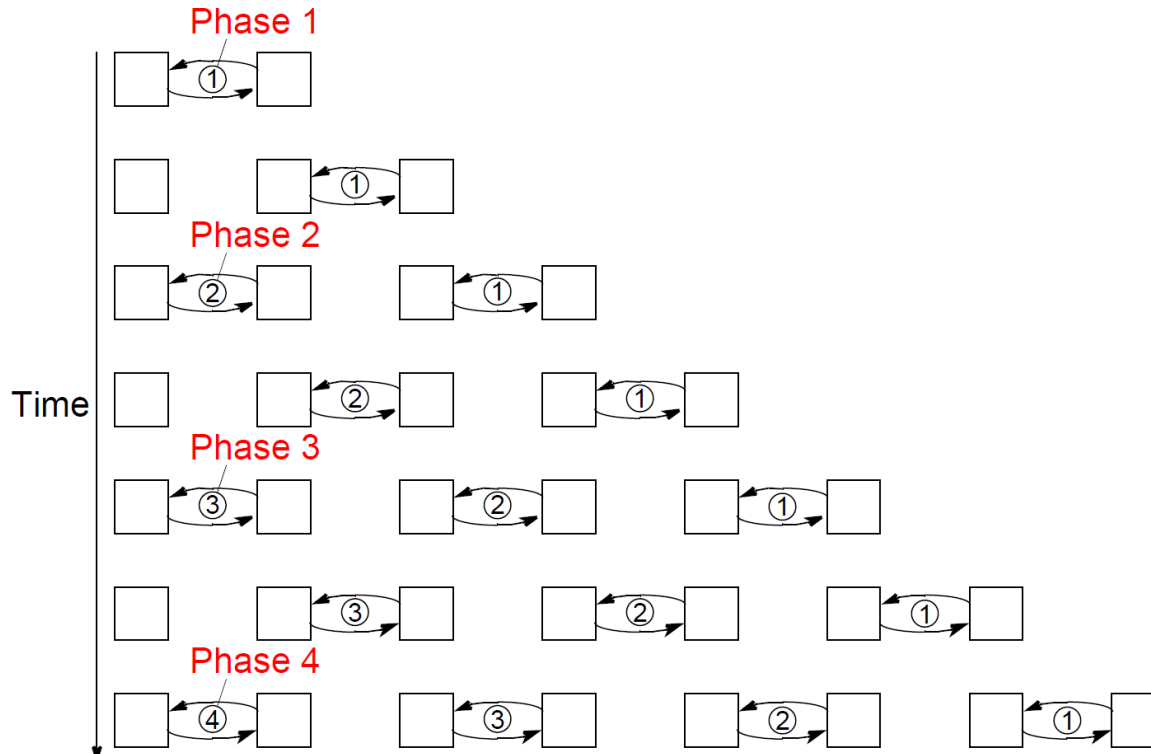
Parallel Bubble Sort

Iteration could start before previous iteration finished if does not overtake previous bubbling action:



Parallel Bubble Sort

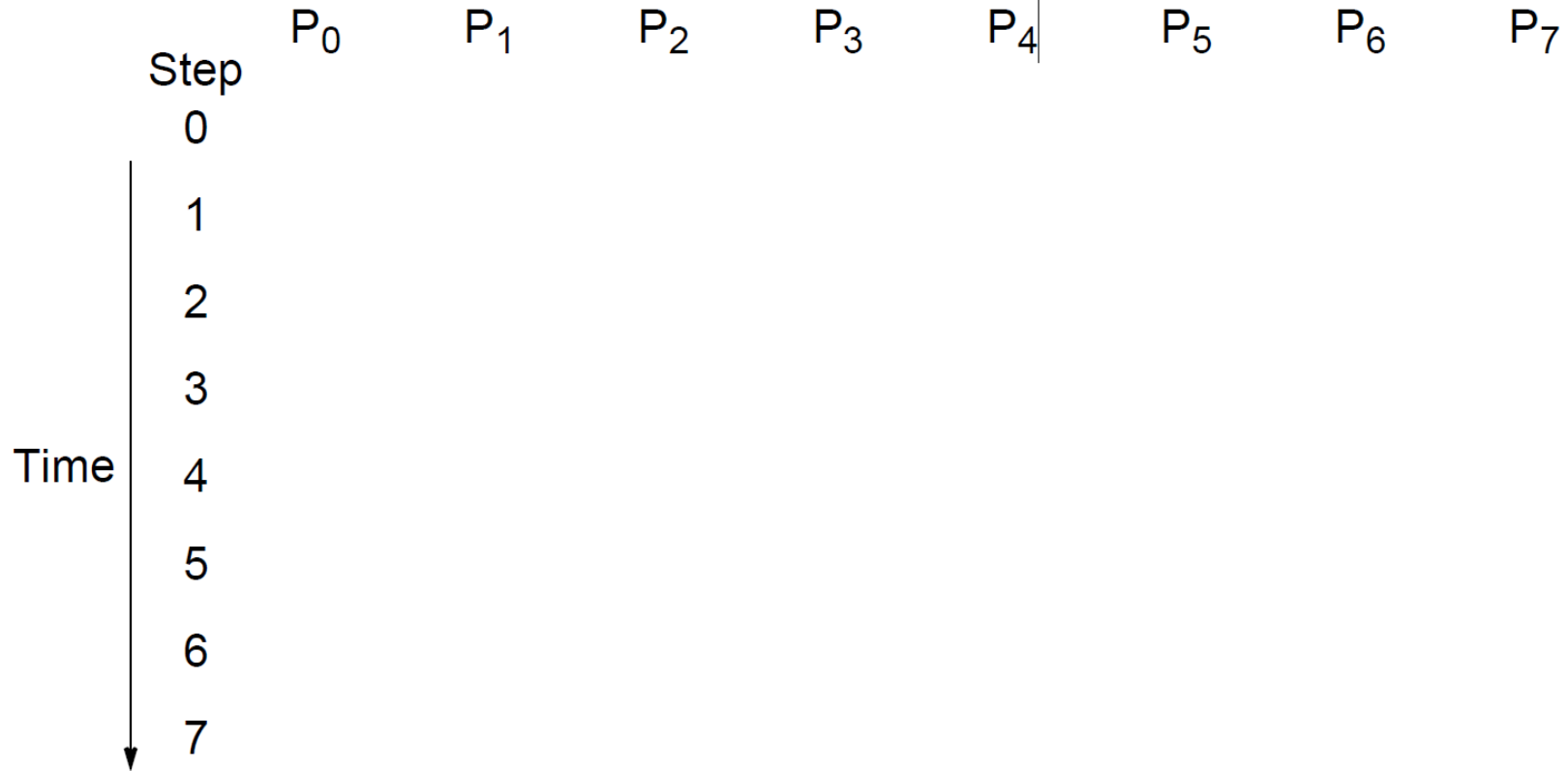
Iteration could start before previous iteration finished if does not overtake previous bubbling action:



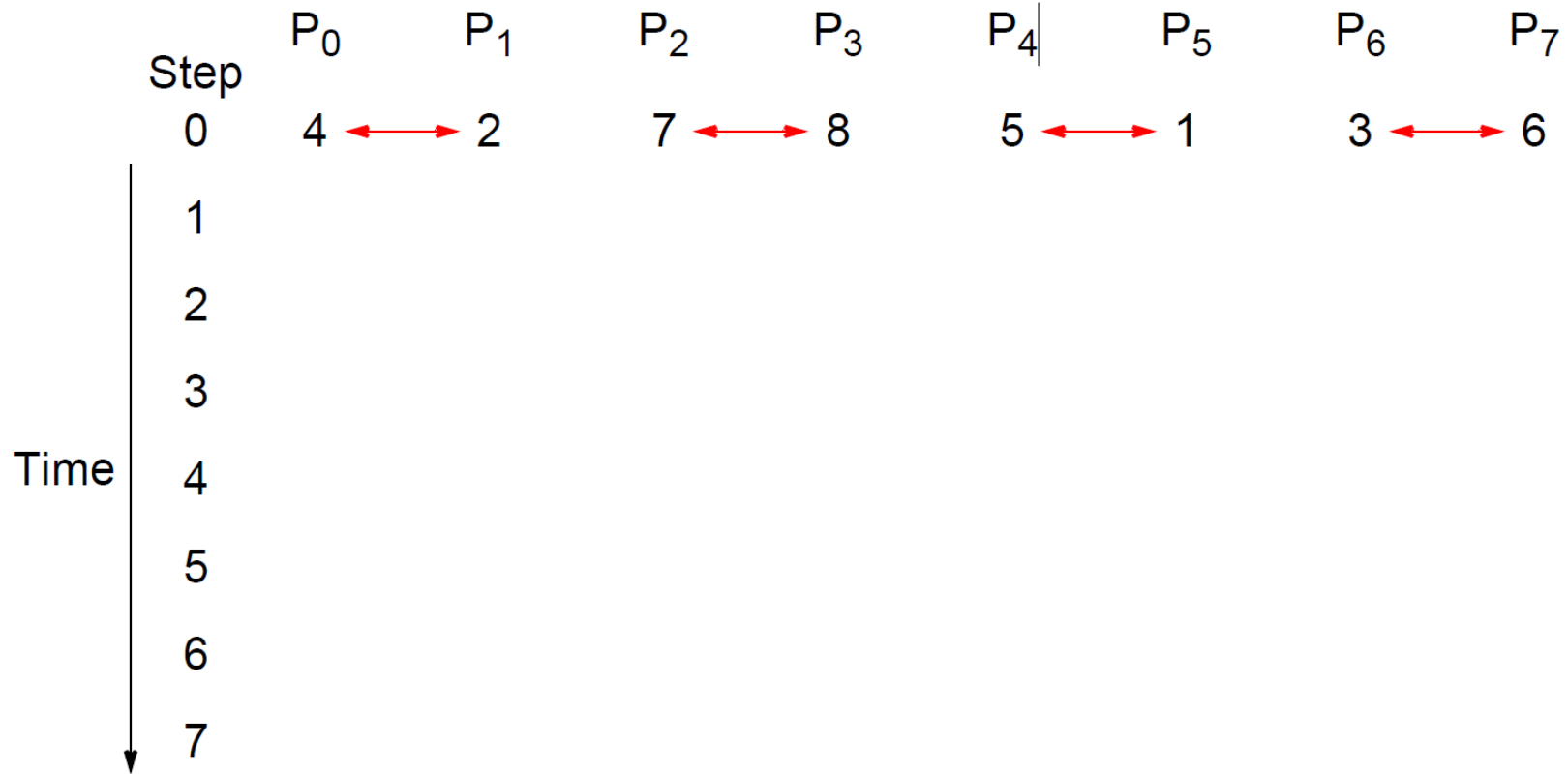
Odd-Even (Transposition) Sort

- Variation of bubble sort.
- Operates in two alternating phases, *even* phase and *odd* phase.
- Even phase: Even-numbered processes exchange numbers with their right neighbor.
- Odd phase: Odd-numbered processes exchange numbers with their right neighbor.

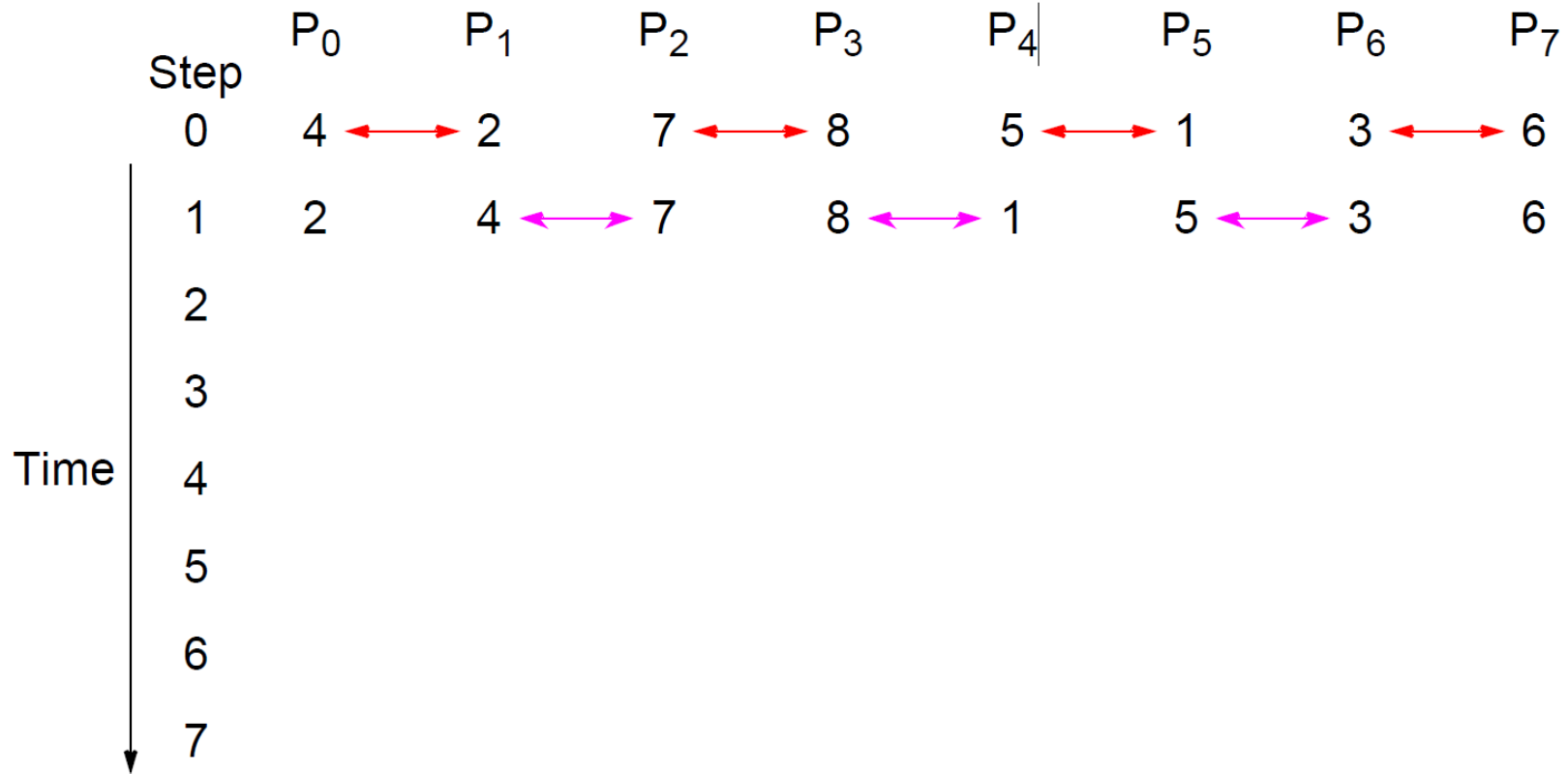
Odd-Even (Transposition) Sort



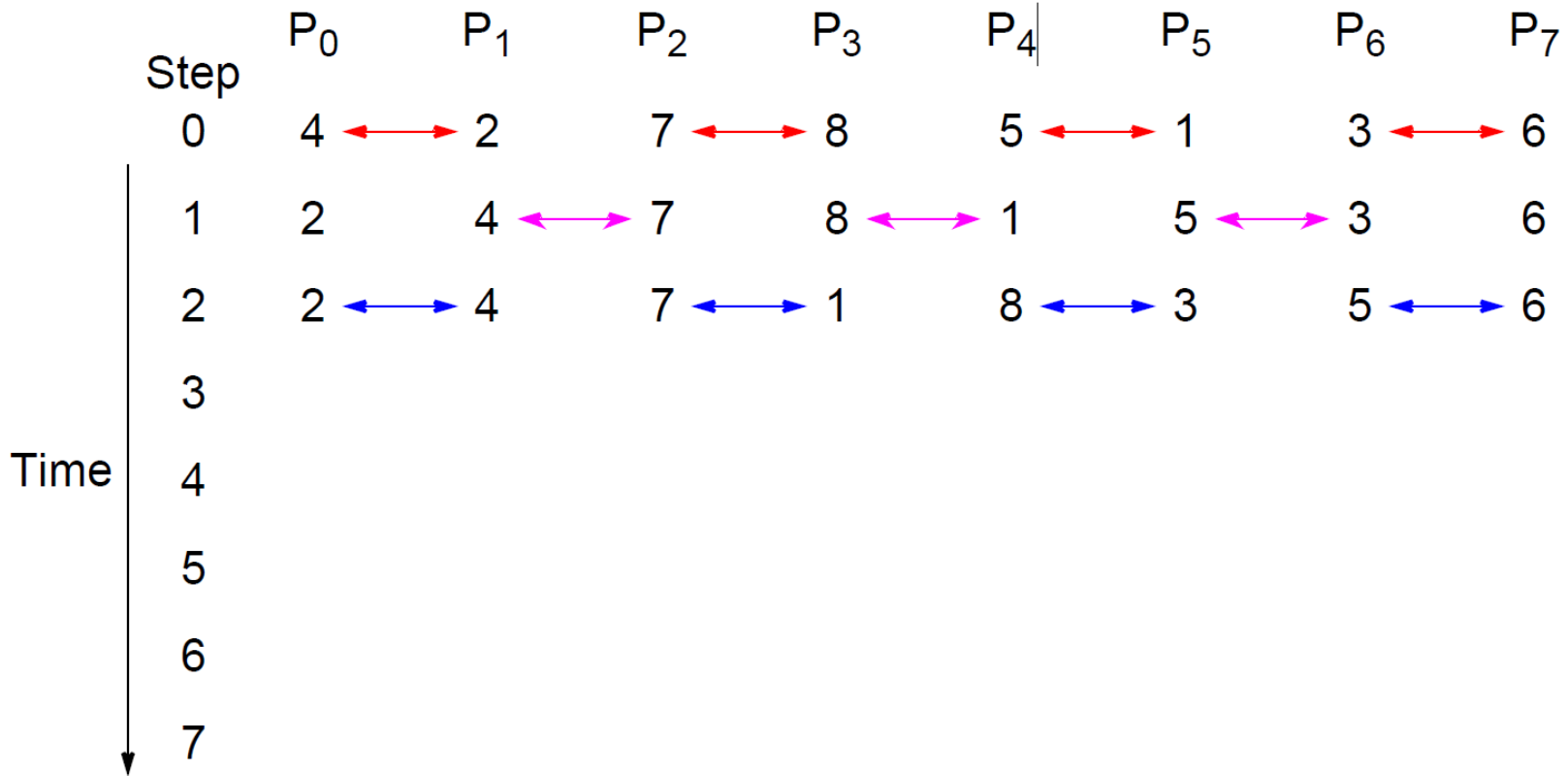
Odd-Even (Transposition) Sort



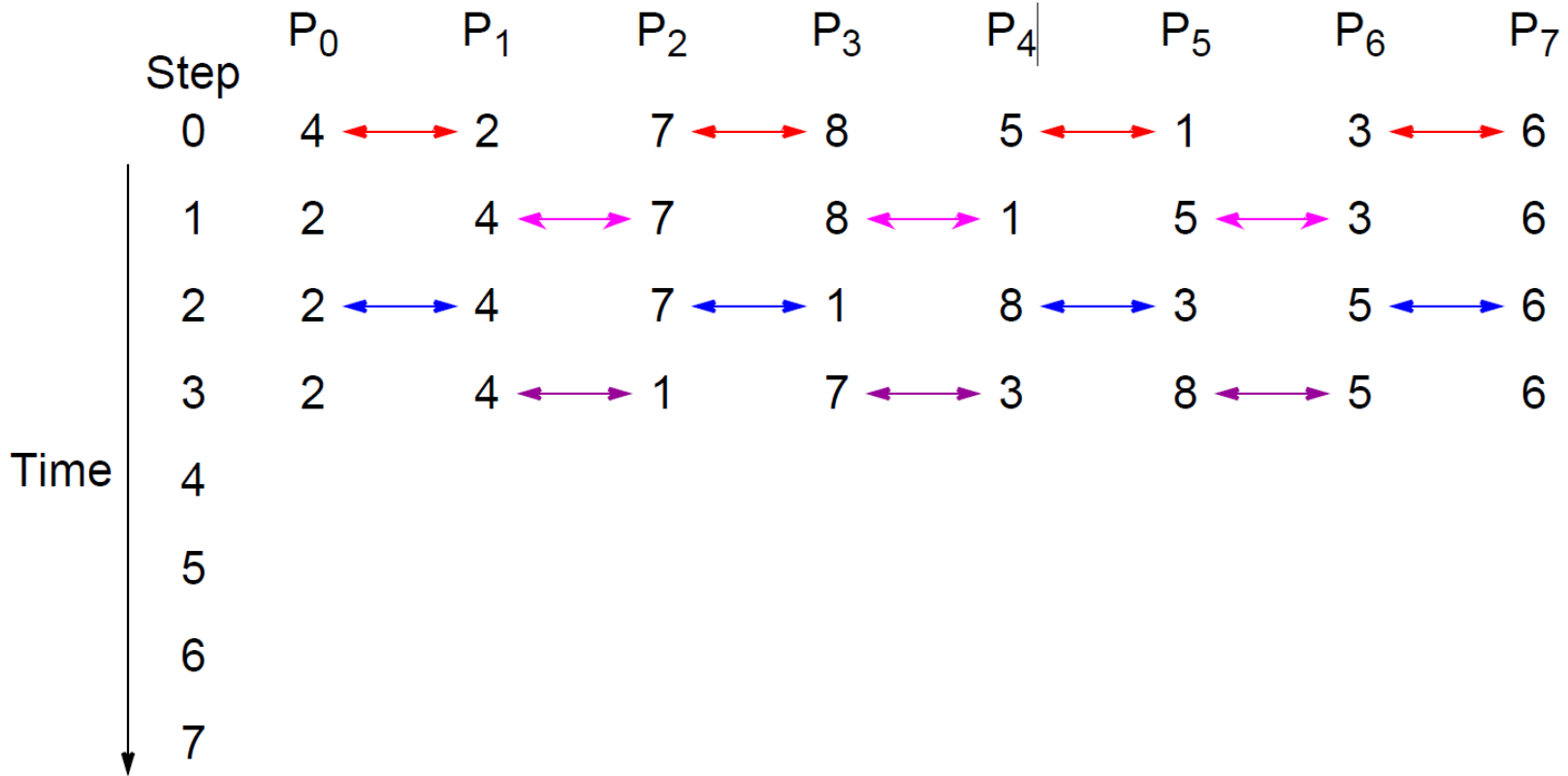
Odd-Even (Transposition) Sort



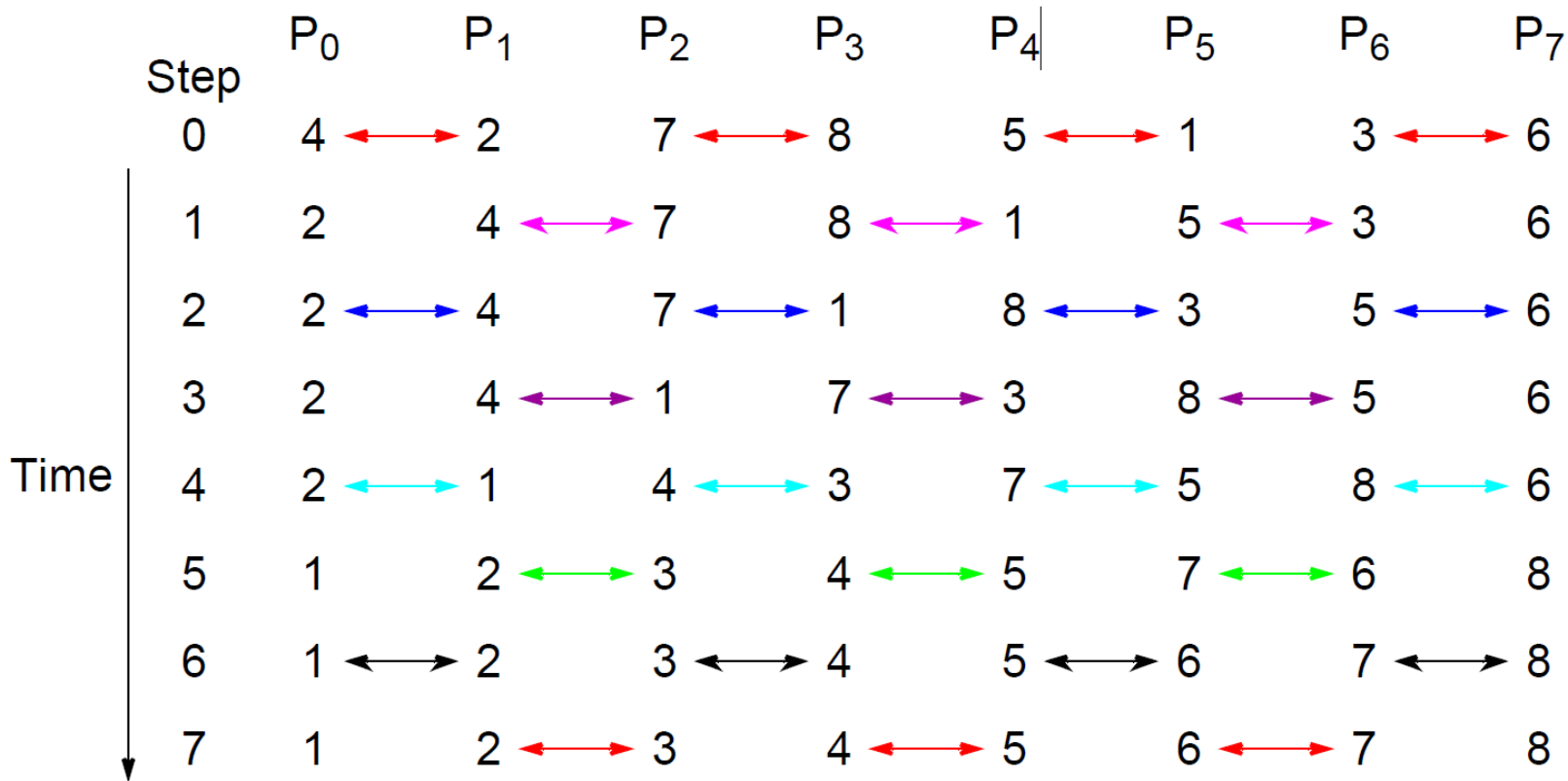
Odd-Even (Transposition) Sort



Odd-Even (Transposition) Sort



Odd-Even (Transposition) Sort



Odd-Even (Transposition) Sort

Even Phase

$P_i, i = 0, 2, 4, \dots, n-2$ (even)

send(&A, P_{i+1});

recv(&B, P_{i+1});

if ($A > B$) $A = B$;

$P_i, i = 1, 3, 5, \dots, n-1$ (odd)

recv(&A, P_{i-1});

send(&B, P_{i-1});

if ($A > B$) $B = A$;

Odd-Even (Transposition) Sort

Odd Phase

$P_i, i = 1, 3, 5, \dots, n-3$ (odd)

send(&A, P_{i+1});

recv(&B, P_{i+1});

if ($A > B$) $A = B$;

$P_i, i = 2, 4, 6, \dots, n-2$ (even)

recv(&A, P_{i-1});

send(&B, P_{i-1});

if ($A > B$) $B = A$;

Odd-Even (Transposition) Sort

In both cases, odd-numbered processes execute their send() routines first, and even-numbered processes execute their recv() routines first. Combining,

$P_i, i = 1, 3, 5, \dots, n-3$ (odd)

```
send(&A, Pi-1);  
recv(&B, Pi-1);  
if (A < B) A = B;  
if (i <= n-3) {  
    send(&A, Pi+1);  
    recv(&B, Pi+1);  
    if (A > B) A = B;  
}
```

$P_i, i = 0, 2, 4, \dots, n-2$ (even)

```
recv(&A, Pi+1);  
send(&B, Pi+1);  
if (A < B) B = A;  
if (i >= 2) {  
    recv(&A, Pi-1);  
    send(&B, Pi-1);  
    if (A > B) B = A;  
}
```

Mergesort

A classical sequential sorting algorithm using a divide-and-conquer approach.

divide

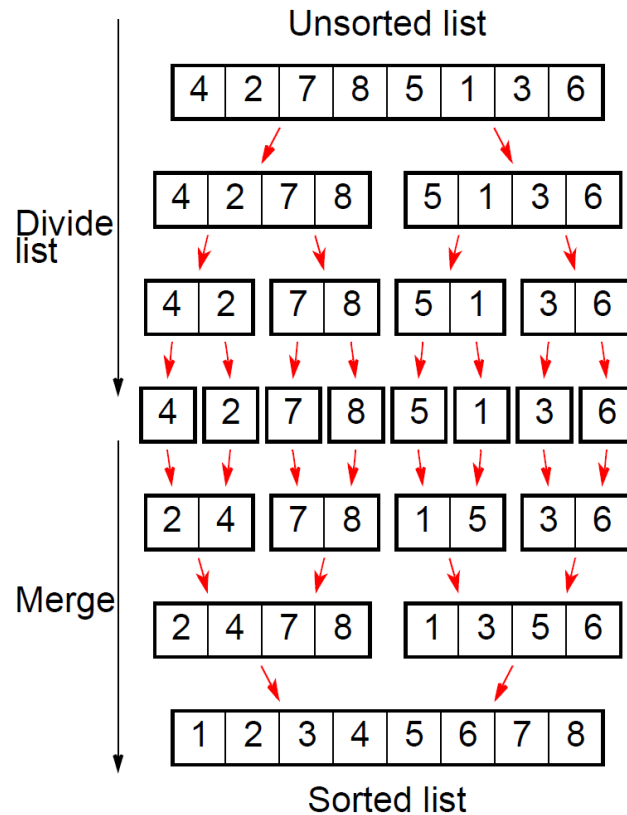
- Unsorted list first divided into half.
- Each half is again divided into two.
- Continued until individual numbers are obtained.

merge

- Then pairs of numbers are combined (merged) into a sorted list of two numbers.
- Pairs of these lists of four numbers are merged into sorted lists of eight numbers.
- This is continued until a fully sorted list is obtained.

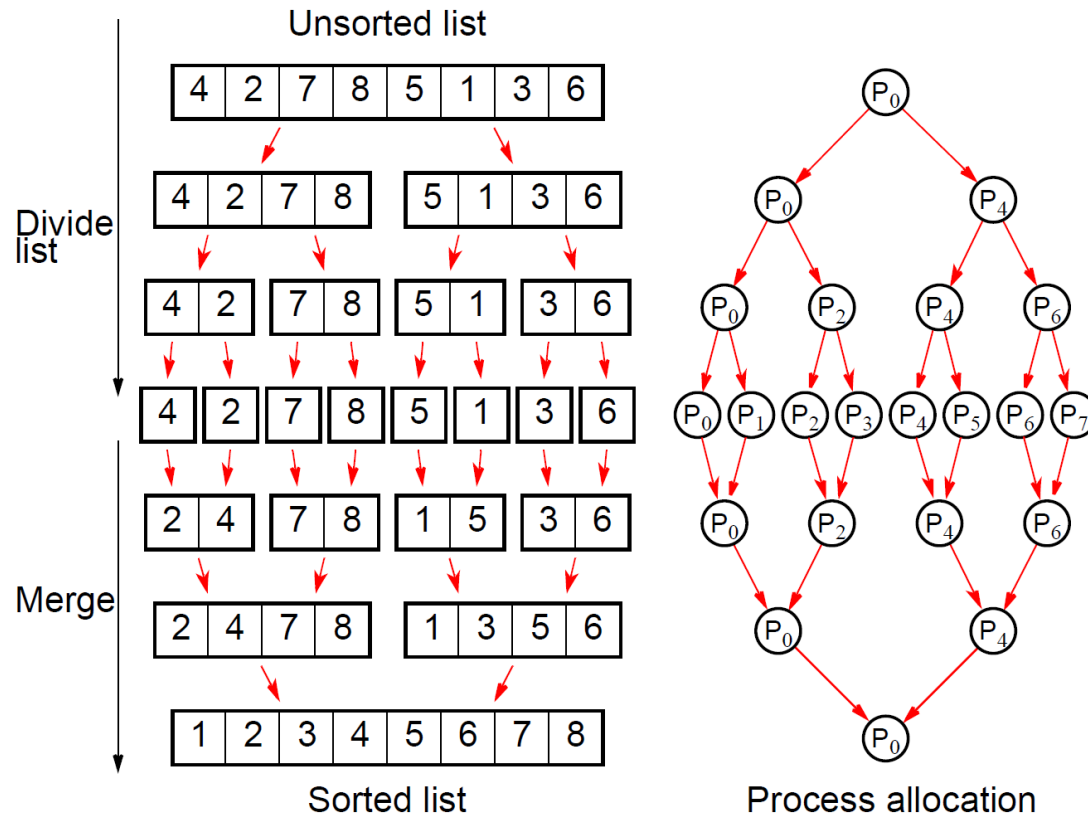
Mergesort

Using tree allocation of processes



Mergesort

Using tree allocation of processes

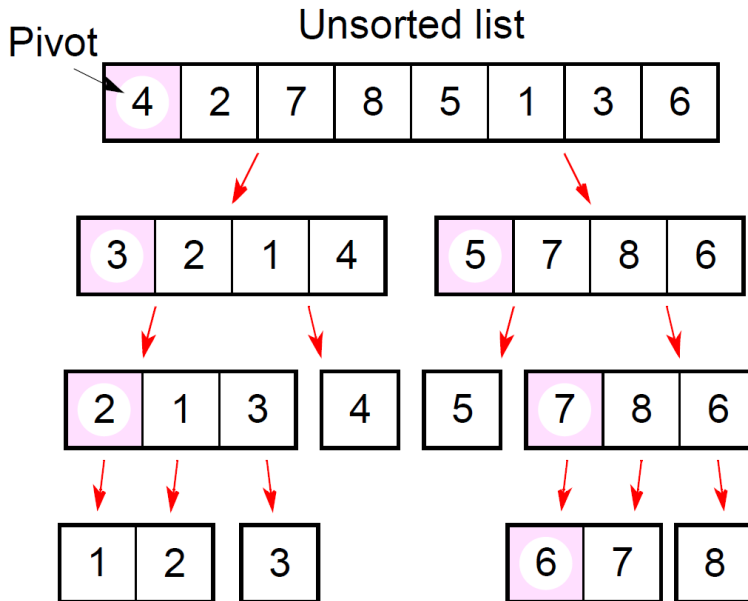


Quicksort

- Popular sequential sorting algorithm with an average sequential time complexity of $O(n \log n)$.
- First list divided into two sublists.
- All the numbers in one sublist arranged to be smaller than all the numbers in the other sublist.
 - Achieved by first selecting one number, called a pivot, k , against which every other number is compared.
 - If the number $< k$, it is placed in one sublist; otherwise, in the other sublist.
 - Pivot could be any number in the list, but often the first number in the list is chosen.
 - Pivot itself could be placed in one sublist, or the pivot could be separated and placed in its final position.
- Continue until sublist becomes a singleton list.

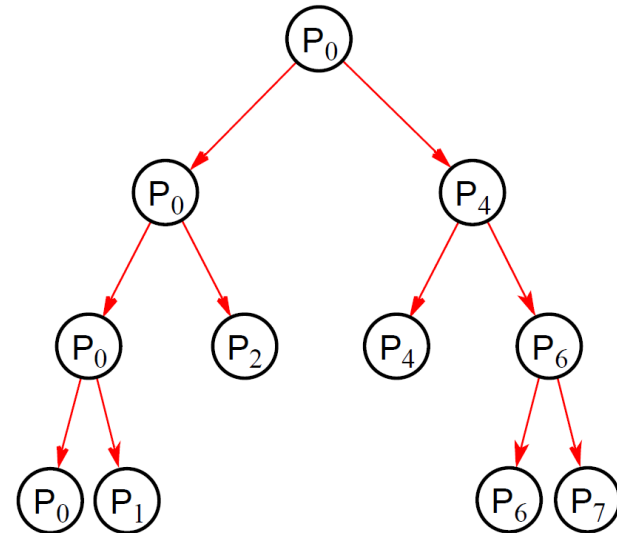
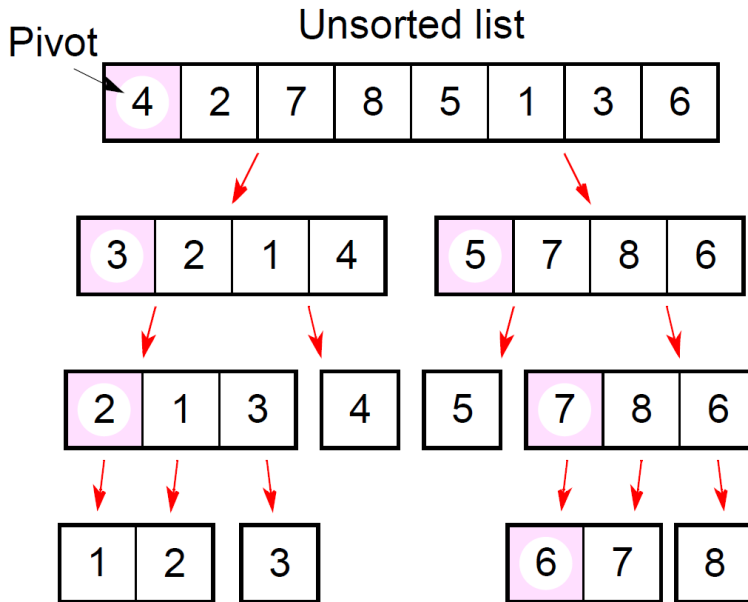
Quicksort

Using tree allocation of processes



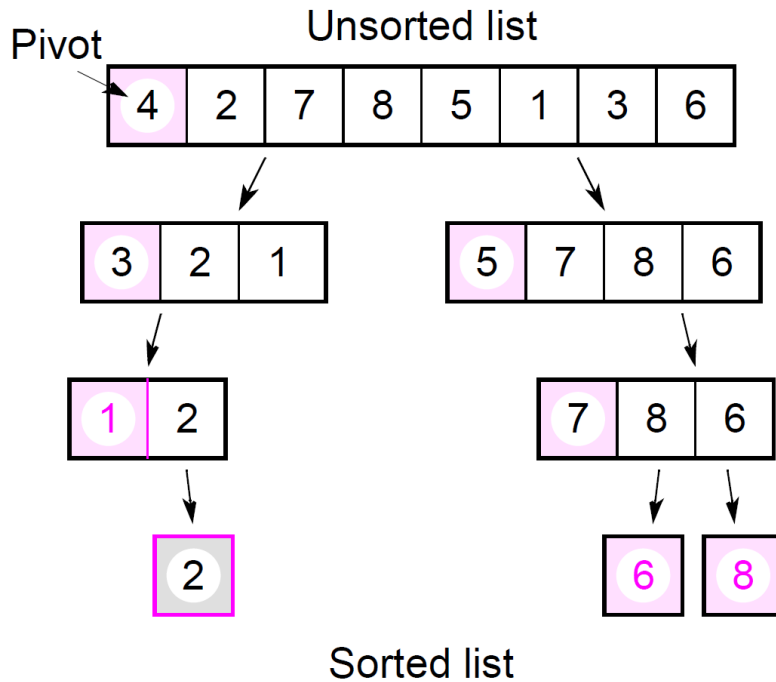
Quicksort

Using tree allocation of processes



Quicksort

With the pivot being withheld in processes:



Quicksort

With the pivot being withheld in processes:

